



Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC)

Departamento de Engenharia e Tecnologias

Base de Dados II

Licenciatura em Engenharia Informática — 2025/2026

Lab #02 — Exploração de Banco de Dados Distribuídos

PARTE 1 — Relatório de Exploração Guiada

Líder do grupo	Nzola Kiampava
Membros do grupo	Eduarda Malungo, Elizandra Manuel, Nzola Kiampava
Turma	EINF6_T2
Link do repositório GitHub	NzolaKiampava/banco_de_dados_distribuido: Exploração de Banco de Dados Distruidos

Índice

1. Fase 0 — Criação do Dataset MercadoKwanza
2. Fase 1 — Docker e Nó Distribuído
3. Fase 2 — Arquitectura Distribuída e Fragmentação
4. Fase 3 — Replicação MySQL: Master e Slaves
5. Fase 4 — Transações, ACID e Two-Phase Commit
6. Registo Consolidado de Uso de IA
7. Dificuldades Encontradas

Introdução

Este relatório documenta os resultados, evidências e reflexões obtidos durante a execução da Parte 1 (Fases 0 a 4) do Lab #02 — Exploração de Banco de Dados Distribuídos, no âmbito da disciplina de Base de Dados II do ISPTEC (Licenciatura em Engenharia Informática, ano lectivo 2025/2026).

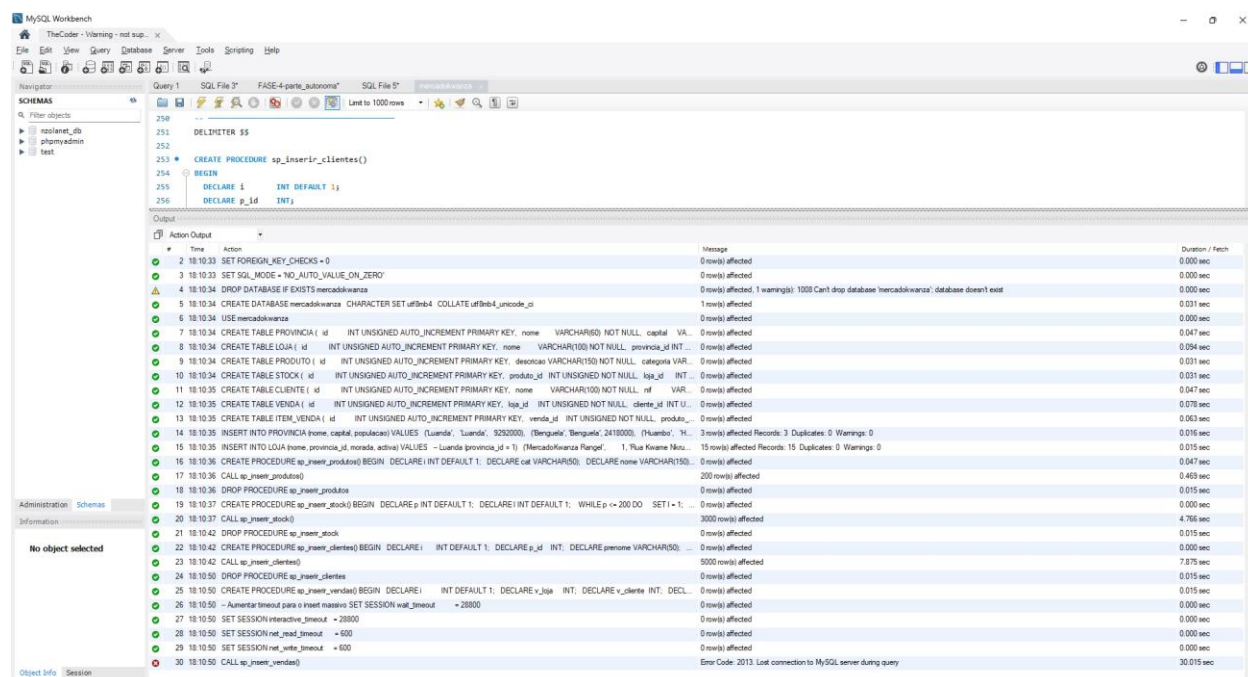
O ambiente distribuído foi montado com Docker, simulando três nós MySQL 8.0 (Luanda, Benguela e Huambo) e um nó MongoDB, sobre o dataset MercadoKwanza, gerado com apoio de IA conforme especificado no enunciado do laboratório.

1. Fase 0 — Criação do Dataset MercadoKwanza

1.1 Descrição do Trabalho Realizado

O script mercadokwanza.sql foi gerado com apoio de IA (Claude Sonnet), conforme o prompt sugerido no enunciado, criando o schema mercadokwanza com as 7 tabelas (PROVINCIA, LOJA, PRODUTO, STOCK, CLIENTE, VENDA, ITEM_VENDA), com chaves primárias AUTO_INCREMENT, chaves estrangeiras e índices nas colunas mais usadas em filtros.

Durante os testes, o script inicial apresentou o erro ERROR CODE: 2013 — Lost Connection ao executar a stored procedure de inserção de vendas, devido ao elevado número de iterações (30 000) com SELECTs individuais dentro do loop. O script foi corrigido substituindo os loops WHILE por operações de INSERT em lote (CROSS JOIN / INSERT...SELECT), o que resolveu o problema de timeout.



Registo de Resultados — FASE-0

Item	Resultado
N.º de linhas em VENDA após importação	30000
N.º de linhas em ITEM_VENDA	90000
N.º de clientes importados	5000
N.º de produtos importados	200
Erros encontrados no script da IA (s/n — quantos?)	1

Evidências

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'mercadokwanza' database with tables: cliente, item_venda, loja, produto, provincia, stock, venda. The main editor shows a SQL query in 'Query 1' with the following code:

```
1 • use mercadokwanza;
2
3 • select 'CLIENTE', count(*) from cliente UNION select 'ITEM_VENDA', count(*) from item_venda
4 UNION SELECT 'LOJA', COUNT(*) FROM loja UNION SELECT 'PRODUTO', COUNT(*) FROM PRODUTO
5 UNION SELECT 'PROVINCIA', COUNT(*) FROM PROVINCIA UNION SELECT 'STOCK', COUNT(*) FROM STOCK
6 UNION SELECT 'VENDA', COUNT(*) FROM VENDA;
```

Below the query editor, the 'Result Grid' shows the output of the query:

CLIENTE	count(*)
CLIENTE	5000
ITEM_VENDA	90000
LOJA	15
PRODUTO	200
PROVINCIA	3
STOCK	3000
VENDA	30000

Registo de Uso de IA — Fase 0 — Geração do script mercadokwanza.sql

Ferramenta usada:

CLAUDE SONNET 4.6

Prompt submetido (resumo):

AO EXECUTAR NO MYSQL, APARECE ESSE ERRO: CALL sp_inserir_vendas() ERROR CODE:2013 LOST CONNECTION

O que a IA respondeu (resumo):

Esse erro acontece porque a stored procedure sp_inserir_vendas() demora muito tempo — inserir 30 000 vendas com SELECT dentro do loop é muito lento e o MySQL Workbench/cliente perde a ligação.

A solução é substituir a abordagem de loop lento por **INSERT em batch com valores gerados via SQL puro**, sem stored procedure. Muito mais rápido.

O que o grupo corrigiu, melhorou ou rejeitou:

O sp_inserir_vendas() fazia um SELECT INTO dentro de um loop de 30 000 iterações — para cada venda, ia buscar o preço do produto com um SELECT. Isso gerava 30 000 queries individuais e demorava vários minutos, causando o Lost Connection (2013).

-- No MySQL Workbench: File > Open SQL Script > mercadokwanza.sql

-- Aumentar timeout antes: Edit > Preferences > SQL Editor > DBMS timeout = 600

-- Ou via terminal:

```
mysql -uroot -pkwanza2024 < mercadokwanza.sql
```

Reflexão Escrita — Fase 0

Pergunta 1: O script gerado pela IA funcionou directamente ou precisou de correcções? Que tipo de erros surgiram e o que aprenderam ao corrigi-los?

Ainda gerou erro no próprio Workbench, o que nos levou fazer mais outras pesquisas, afim de executar o script corrigido sem dar erro de timeout.

Com isso aprendemos a aumentar o tempo de execução do no Workbench

Pergunta 2: *Porque é que o grupo decidiu usar stored procedures/operações em lote em vez de INSERT linha a linha? Qual a vantagem em termos de tempo e manutenção?*

A escolha de **stored procedures** ou **operações em lote (batch inserts)** em vez de **INSERT linha a linha** é feita principalmente por questões de **desempenho, eficiência e manutenção**

1. Vantagem em termos de tempo (performance)

Quando se faz um INSERT linha a linha:

```
INSERT INTO clientes VALUES (...);  
INSERT INTO clientes VALUES (...);  
INSERT INTO clientes VALUES (...);
```

O banco de dados precisa processar cada comando individualmente, realizando várias validações, acessos ao disco e comunicações entre a aplicação e o servidor.

Já com operações em lote:

```
INSERT INTO clientes VALUES (...),  
                        (...),  
                        (...);
```

ou através de uma stored procedure que recebe vários registros, o banco processa tudo de uma só vez.

Benefícios:

- Menos comunicação entre aplicação e banco de dados.
- Menos operações de I/O (leitura/escrita em disco).
- Menor sobrecarga de processamento.
- Redução significativa do tempo de execução quando existem milhares ou milhões de registros.

2. Vantagem em termos de manutenção

Com uma **stored procedure**, a lógica fica centralizada no banco de dados:

```
CALL InserirCliente(...);
```

Em vez de repetir a mesma lógica em várias partes da aplicação.

Benefícios:

- Código mais organizado.
- Alterações são feitas num único local.
- Menor risco de inconsistências.
- Reutilização da lógica por diferentes aplicações ou módulos.

2. Fase 1 — Docker e a Ideia de Nó Distribuído

2.1 Instalação e Configuração do Docker

O Docker Desktop foi instalado num ambiente Windows com WSL2 activado. Devido à porta 3306 já estar ocupada por uma instalação local de MySQL, o nó Luanda (Master) foi remapeado para a porta 3310 no anfitrião (alteração documentada na secção de Dificuldades).

Registo de Resultados — FASE-1A

Item	Resultado
Versão do Docker instalada	29.5.3
Versão do Docker Compose	5.1.4
Sistema Operativo usado pelo grupo	WINDOWS

2.2 Estrutura de Pastas e docker-compose.yml

Foi criada a estrutura de pastas mercadokwanza/ com as subpastas dados/ (contendo mercadokwanza.sql) e scripts/, conforme especificado no enunciado. O ficheiro docker-compose.yml define os 4 contentores: no-luanda (Master MySQL), no-benguela e no-huambo (Slaves MySQL) e mongo-kwanza (MongoDB).

2.3 Levantamento do Cluster

Comandos executados:

```
docker compose up -d
docker ps
```

```
docker exec -it no-luanda mysql -uroot -pkwanza2024
SHOW DATABASES; USE mercadokwanza; SHOW TABLES;
SELECT COUNT(*) FROM VENDA;
SELECT COUNT(*) FROM CLIENTE;
```

Registo de Resultados — FASE-1B

Item	Resultado
N.º de contentores com STATUS Up	4
Tempo de arranque do cluster (aprox.)	21 segundos
Resultado de COUNT(*) FROM VENDA (Luanda)	2250
Resultado de COUNT(*) FROM CLIENTE (Luanda)	5000
Os dados do SQL foram carregados automaticamente? (S/N)	S

Evidências

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Gordon, Containers, Images (selected), Volumes, Kubernetes, Builds, Docker Hub, MCP Toolkit (BETA), Logs, Models, and Extensions. The main area is titled 'Images' and shows a list of locally installed images:

Name	Tag	Image ID	Created	Size	Actions
postgres	16	081f1bc7bd5e	10 days ago	641.57 MB	[Pull] [Run] [More] [Delete]
mongo	7.0	948b635bc126	10 days ago	1.18 GB	[Pull] [Run] [More] [Delete]
mysql	8.0	7dcddc01f13b	2 months ago	1.1 GB	[Pull] [Run] [More] [Delete]

Below the images list, there are 'Walkthroughs' for 'How do I run a container?' (6 mins) and 'Run Docker Hub images' (5 mins). At the bottom, a status bar shows 'Engine running', system resources (RAM 1.80 GB, CPU 2.07%, Disk 6.76 GB used), and a link to 'Update version'.


```
Windows PowerShell (x86)
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
83bb12d90b35   mysql:8.0   "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp   no-huambo
f3ecaf8fe1dd   mongo:7.0   "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp   mongo-kwanza
6f82571ce86c   mysql:8.0   "docker-entrypoint.s..." About a minute ago Up About a minute   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp   no-benguela
3ab829f64457   mysql:8.0   "docker-entrypoint.s..." About a minute ago Up 25 seconds       0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   no-luanda

PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-luanda mysql -uroot -pkwanza2024 -e "USE mercadoKwanza; SHOW TABLES;"
mysql: [Warning] Using a password on the command line interface can be insecure.

+-----+
| Tables_in_mercadoKwanza |
+-----+
| CLIENTE                  |
| ITEM_VENDA               |
| LOJA                     |
| PRODUTO                  |
| PROVINCIA                |
| STOCK                    |
| VENDA                    |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-luanda
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-luanda mysql -uroot -pkwanza2024 -e "USE mercadoKwanza; SELECT 'PROVINCIA', COUNT(*) FROM PROVINCIA UNION ALL SELECT 'LOJA', COUNT(*) FROM LOJA UNION ALL SELECT 'PRODUTO', COUNT(*) FROM PRODUTO UNION ALL SELECT 'CLIENTE', COUNT(*) FROM CLIENTE UNION ALL SELECT 'VENDA', COUNT(*) FROM VENDA UNION ALL SELECT 'ITEM_VENDA', COUNT(*) FROM ITEM_VENDA;"
mysql: [Warning] Using a password on the command line interface can be insecure.

+-----+-----+
| PROVINCIA | COUNT(*) |
+-----+-----+
| PROVINCIA |        3 |
| LOJA      |       15 |
| PRODUTO   |      200 |
| CLIENTE   |     5000 |
| VENDA     |      7375 |
| ITEM_VENDA |    22124 |
+-----+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-luanda
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza>
```

```
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker compose up -d
time="2026-06-20T19:40:48+01:00" level=warning msg="C:\\Users\\Nzola Kiampava\\Documents\\mercadoKwanza\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 5/5
✔ Network mercadoKwanza_default Created 0.1s
✔ Container no-luanda Started 1.2s
✔ Container no-benguela Started 1.2s
✔ Container no-huambo Started 1.2s
✔ Container mongo-kwanza Started 1.2s

What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=mercadoKwanza
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker compose ps
time="2026-06-20T19:41:11+01:00" level=warning msg="C:\\Users\\Nzola Kiampava\\Documents\\mercadoKwanza\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE      COMMAND                  SERVICE    CREATED        STATUS        PORTS
mongo-kwanza        mongo:7.0   "docker-entrypoint.s..." mongod      22 seconds ago Up 21 seconds  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
no-benguela         mysql:8.0   "docker-entrypoint.s..." mysql-benguela 22 seconds ago Up 21 seconds  0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
no-huambo           mysql:8.0   "docker-entrypoint.s..." mysql-huambo  22 seconds ago Up 21 seconds  0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp
no-luanda           mysql:8.0   "docker-entrypoint.s..." mysql-luanda  22 seconds ago Up 21 seconds  0.0.0.0:3310->3306/tcp, [::]:3310->3306/tcp
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza>
```

Reflexão Escrita — Fase 1

Pergunta 1: O que significa o mapeamento de portas '3307:3306' (ou, no caso deste grupo, '3310:3306' para Luanda)? Porque é que os nós Benguela e Huambo não podem também usar a porta 3306 no anfitrião?

O mapeamento 3307:3306 significa que a porta 3307 do computador anfitrião (o teu Windows) é redirecionada para a porta 3306 dentro do contentor. É como dizer: "quem bater na porta 3307 cá fora, entra pela porta 3306 lá dentro".

No caso deste grupo, o nó Luanda usa 3310:3306 porque a porta 3306 do anfitrião já estava ocupada pelo MySQL instalado localmente no Windows.

Os nós Benguela e Huambo não podem usar a mesma porta no anfitrião porque uma porta só pode estar associada a um processo de cada vez. É como um número de telefone — duas pessoas não podem ter o mesmo número simultaneamente. Por isso cada nó tem a sua porta no anfitrião (3310, 3307, 3308), mas internamente todos correm na porta padrão do MySQL (3306) dentro do seu próprio contentor — sem conflito, porque cada contentor é um ambiente isolado.

Pergunta 2: Qual a vantagem de usar o volume `./dados:/docker-entrypoint-initdb.d`? O que aconteceria se não usassem esse volume e quisessem carregar os dados manualmente?

permite que os ficheiros SQL armazenados na pasta dados do computador sejam automaticamente executados quando o contentor do MySQL é criado pela primeira vez.

Sem esse volume, seria necessário importar os dados manualmente sempre que o ambiente fosse recriado, aumentando o tempo de configuração e a possibilidade de erros.

Pergunta 3: Se desligarem o computador e voltarem a ligar, os dados permanecem? Testem e documentem o que observam.

Após desligar e voltar a ligar o computador, os dados permaneceram disponíveis porque o contentor não foi removido e o Docker preserva o sistema de ficheiros do contentor no disco. Contudo, verificou-se que a configuração atual não utiliza um volume dedicado para `/var/lib/mysql`; portanto, caso o contentor seja removido e recriado, os dados poderão ser perdidos. Para garantir persistência permanente,

recomenda-se a utilização de um volume Docker para armazenar os ficheiros da base de dados.

3. Fase 2 — Arquitectura Distribuída e Fragmentação

3.1 Fragmentação Horizontal de VENDA

Foram criadas as views frag_venda_luanda, frag_venda_benguela e frag_venda_huambo, filtrando as vendas pela província da loja associada (via JOIN com LOJA).

Comandos executados (nó Luanda):

```
CREATE OR REPLACE VIEW frag_venda_luanda AS ...
CREATE OR REPLACE VIEW frag_venda_benguela AS ...
CREATE OR REPLACE VIEW frag_venda_huambo AS ...
SELECT 'Luanda', COUNT(*) FROM frag_venda_luanda UNION ALL ...
```

3.2 Fragmentação Vertical de PRODUTO

Foi proposta uma fragmentação vertical da tabela PRODUTO, separando os atributos de catálogo (descricao, categoria) dos atributos comerciais (preco, activo).

Query proposta pelo grupo:

```
CREATE OR REPLACE VIEW produto_catálogo AS
SELECT id, descricao, categoria FROM PRODUTO;
```

```
CREATE OR REPLACE VIEW produto_comercial AS
SELECT id, preco, activo FROM PRODUTO;
```

Registo de Resultados — FASE-2

Item	Resultado
Total vendas — fragmento Luanda	9984
Total vendas — fragmento Benguela	10026
Total vendas — fragmento Huambo	9990
Soma dos 3 fragmentos = total? (S/N)	30000
Query de fragmentação vertical escrita pelo grupo? (S/N)	S

Evidências

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
19 #Verifica a completude (soma deve ser igual ao total):
20 • SELECT 'Luanda' AS provincia, COUNT(*) AS total FROM frag_venda_luanda
21 UNION ALL
22 SELECT 'Benguela', COUNT(*) FROM frag_venda_benguela
23 UNION ALL
24 SELECT 'Huambo', COUNT(*) FROM frag_venda_huambo
25 UNION ALL
26 SELECT 'TOTAL GERAL', COUNT(*) FROM VENDA;
27
28
29
30 # Cria a fragmentação vertical de PRODUTO:
31
```

Below the query, the 'Result Grid' is displayed with the following data:

provincia	total
Luanda	9984
Benguela	10026
Huambo	9990
TOTAL GERAL	30000

Reflexão Escrita — Fase 2

Pergunta 1: Segundo Özsu & Valduriez (2020), um esquema de fragmentação válido deve respeitar três propriedades: completude, disjunção e reconstituição. As views que criaram respeitam estas três propriedades? Justifiquem para cada uma.

Sim, as views respeitam as três propriedades:

Completude — cada venda da tabela VENDA pertence a uma loja, e cada loja pertence a uma província (1, 2 ou 3). Como os três fragmentos cobrem todas as províncias, nenhuma venda fica de fora. Toda a informação está representada em algum fragmento.

Disjunção — uma loja só pode pertencer a uma província, logo uma venda só pode aparecer num único fragmento. Não existe sobreposição entre os três fragmentos — o mesmo registo nunca aparece em dois fragmentos ao mesmo tempo.

Reconstituição — a tabela original VENDA pode ser reconstruída fazendo UNION ALL dos três fragmentos, como demonstrado na query de verificação. O resultado da soma dos três fragmentos é igual ao COUNT(*) total da tabela VENDA.

Pergunta 2: *Quais são as limitações de simular fragmentação com VIEWS em vez de distribuir fisicamente os dados por diferentes servidores? O que é que as VIEWS não conseguem simular?*

A utilização de VIEWS permite simular a fragmentação lógica dos dados, mas não a sua distribuição física. Todos os dados permanecem armazenados no mesmo servidor, não existindo comunicação de rede, processamento distribuído, replicação, tolerância a falhas ou custos de transferência entre nós. Assim, as VIEWS são úteis para estudar os conceitos de fragmentação, mas não conseguem reproduzir o comportamento completo de uma base de dados distribuída real.

Pergunta 3: *Num cenário real, que factores decidem se uma tabela deve ser fragmentada horizontalmente, verticalmente, ou não fragmentada de todo?*

Padrão de acesso — se cada região acede maioritariamente aos seus próprios dados (ex: lojas de Benguela consultam quase sempre vendas de Benguela), a fragmentação horizontal por região faz sentido. Se diferentes aplicações acedem a subconjuntos de colunas (ex: o sistema de catálogo só precisa de descrição e categoria, nunca do preço), a fragmentação vertical é mais adequada.

Volume de dados — tabelas muito grandes como VENDA e ITEM_VENDA beneficiam de fragmentação horizontal porque cada nó trabalha com um subconjunto menor, reduzindo o tempo de resposta nas queries locais.

Frequência de actualização — tabelas de referência pequenas e estáveis como PROVINCIA e PRODUTO são melhores candidatas a replicação total em todos os nós do que a fragmentação, porque são consultadas por todos e raramente mudam.

Localidade dos dados — se existe uma relação clara entre os dados e uma localização física (ex: vendas de Luanda processadas em Luanda), a fragmentação horizontal por essa dimensão reduz a latência e o tráfego de rede.

Joins frequentes — se uma tabela é frequentemente combinada com outra em JOIN, fragmentar as duas pelo mesmo critério (co-localização) evita joins distribuídos que são muito mais lentos do que joins locais.

No caso do MercadoKwanza, VENDA e STOCK são boas candidatas a fragmentação horizontal por província, enquanto PRODUTO e PROVINCIA devem existir replicadas em todos os nós por serem dados de referência consultados globalmente.

4. Fase 3 — Replicação MySQL: Master e Slaves

4.1 Configuração da Replicação

A replicação Master-Slave foi configurada a partir das coordenadas do binary log do nó Luanda (Master), aplicadas nos nós Benguela e Huambo (Slaves) via CHANGE MASTER TO / START SLAVE.

Comandos executados:

```
docker exec -it no-luanda mysql -uroot -pkwanza2024 -e "SHOW MASTER STATUS\G"
-- No Slave (Benguela e Huambo):
CHANGE MASTER TO MASTER_HOST='no-luanda', MASTER_USER='root',
  MASTER_PASSWORD='kwanza2024', MASTER_LOG_FILE='...', MASTER_LOG_POS=...;
START SLAVE;
SHOW SLAVE STATUS\G
```

Registo de Resultados — FASE-3 (Replicação)

Item	Resultado
Slave_IO_Running — Benguela	Yes
Slave_SQL_Running — Benguela	Yes
Seconds_Behind_Master — Benguela	0
Slave_IO_Running — Huambo	Yes
Seconds_Behind_Master — Huambo	0
Os 3 produtos inseridos aparecem no Slave Benguela? (S/N)	S
Os 3 produtos inseridos aparecem no Slave Huambo? (S/N)	S


```

PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> Get-Content "C:\Users\Nzola Kiampava\Documents\mercadoKwanza\dados\dump.sql" | docker exec -i no-huambo mysql -uroot -pkwanza2024 mercadoKwanza
mysql: [Warning] Using a password on the command line interface can be insecure.
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> Get-Content "C:\Users\Nzola Kiampava\Documents\mercadoKwanza\dados\dump.sql" | docker exec -i no-benguela mysql -uroot -pkwanza2024 mercadoKwanza
mysql: [Warning] Using a password on the command line interface can be insecure.
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadoKwanza -e "SELECT id, descricao, preco FROM PRODUTO ORDER BY id DESC LIMIT 5;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+-----+-----+
| id | descricao | preco |
+-----+-----+-----+
| 209 | Pilha AA Energizer x4 | 1200.00 |
| 208 | Arroz Precioso 5kg | 3500.00 |
| 207 | Sab????o Protex 200g | 850.00 |
| 206 | Pilha AA Energizer x4 | 1200.00 |
| 205 | Arroz Precioso 5kg | 3500.00 |
+-----+-----+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-benguela
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-huambo mysql -uroot -pkwanza2024 mercadoKwanza -e "SELECT id, descricao, preco FROM PRODUTO ORDER BY id DESC LIMIT 5;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+-----+-----+
| id | descricao | preco |
+-----+-----+-----+
| 209 | Pilha AA Energizer x4 | 1200.00 |
| 208 | Arroz Precioso 5kg | 3500.00 |
| 207 | Sab????o Protex 200g | 850.00 |
| 206 | Pilha AA Energizer x4 | 1200.00 |
| 205 | Arroz Precioso 5kg | 3500.00 |
+-----+-----+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-huambo
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> |

```

4.2 Simulação de Falha do Master

Comandos executados:

```

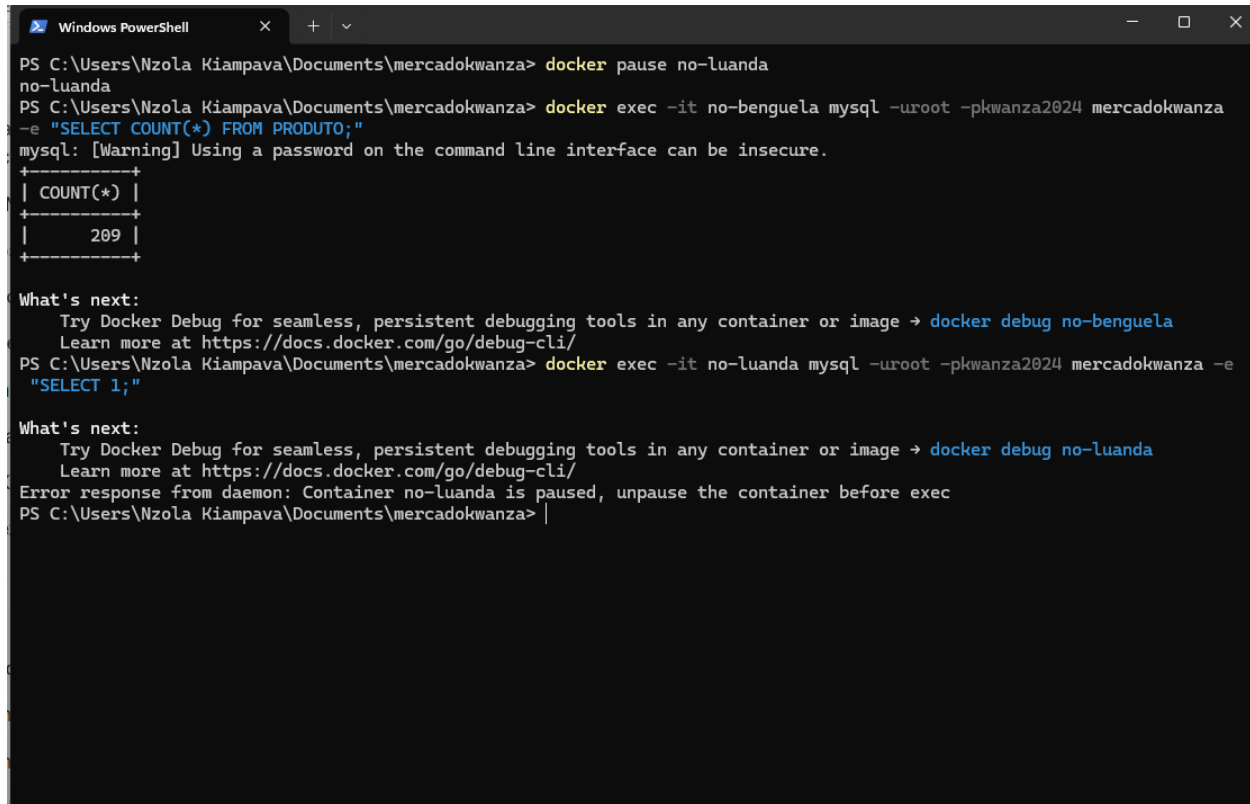
docker pause no-luanda
docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadoKwanza -e "SELECT 1;"
docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadoKwanza -e "SELECT COUNT(*) FROM PRODUTO;"
docker unpause no-luanda

```

Registo de Resultados — FASE-3B (Falha do Master)

Item	Resultado
O que aconteceu às leituras no Slave quando o Master estava pausado?	continuaram a funcionar normalmente
O que aconteceu quando tentaram escrever com o Master pausado?	comando ficou sem respost
Após restaurar o Master, o Slave sincronizou automaticamente? (S/N)	Sim, automática

Evidências



```
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker pause no-luanda
no-luanda
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadoKwanza
-e "SELECT COUNT(*) FROM PRODUTO;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| COUNT(*) |
+-----+
|      209 |
+-----+

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-benguela
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadoKwanza -e
"SELECT 1;"

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-luanda
  Learn more at https://docs.docker.com/go/debug-cli/
Error response from daemon: Container no-luanda is paused, unpause the container before exec
PS C:\Users\Nzola Kiampava\Documents\mercadoKwanza> |
```

Registro de Uso de IA — Fase 3 — Apoio na configuração de replicação (se aplicável)

Ferramenta usada:

Claude sonnet 4.0

Prompt submetido (resumo):

Configuração de replicação MySQL Master-Slave em Docker; resolução do erro de autenticação caching_sha2_password; comandos CHANGE MASTER TO e SHOW SLAVE STATUS.

O que a IA respondeu (resumo):

Criação de utilizador replicador com `mysql_native_password` para contornar o erro de SSL; comandos completos de STOP SLAVE, RESET SLAVE e CHANGE MASTER TO com os valores correctos do binary log (File e Position obtidos do SHOW MASTER STATUS).

O que o grupo corrigiu, melhorou ou rejeitou:

Foi necessário adaptar os comandos do CMD para PowerShell (remoção do `2>nul`). Foi também necessário fazer dump e reimportação dos dados no Huambo e Benguela porque a replicação não copia dados históricos, apenas operações futuras.

Reflexão Escrita — Fase 3

Pergunta 1: *A replicação Master-Slave é síncrona ou assíncrona? O que significa isso para a consistência dos dados? Se um cliente comprar um produto em Luanda (Master) e imediatamente consultar o stock em Benguela (Slave), pode ver um valor desactualizado?*

A replicação MySQL Master-Slave é **assíncrona** — o Master confirma o COMMIT ao cliente sem esperar que os Slaves apliquem a operação. Isso significa que existe um intervalo de tempo (lag) durante o qual os Slaves podem ter dados ligeiramente desactualizados. Sim, um cliente que compre um produto em Luanda e consulte imediatamente o stock em Benguela pode ver o valor anterior à compra, especialmente se o `Seconds_Behind_Master` for maior que zero.

Pergunta 2: *Qual o papel do binary log (binlog) na replicação? O que aconteceria se o binary log fosse apagado acidentalmente no Master enquanto os Slaves ainda não tinham lido todas as entradas?*

O binary log regista sequencialmente todas as operações de escrita (INSERT, UPDATE, DELETE) executadas no Master. Os Slaves lêem esse log e reproduzem as mesmas operações localmente. Se o binary log fosse apagado acidentalmente enquanto os Slaves ainda não tinham lido todas as entradas, esses Slaves ficariam impossibilitados de sincronizar — perderiam as operações em falta e ficariam com dados inconsistentes, exigindo uma reimportação completa a partir do Master.

Pergunta 3: *Numa arquitectura real com 10 Slaves, o que acontece se o Master cair permanentemente? Que estratégia poderia o grupo adoptar para tornar o sistema mais resiliente?*

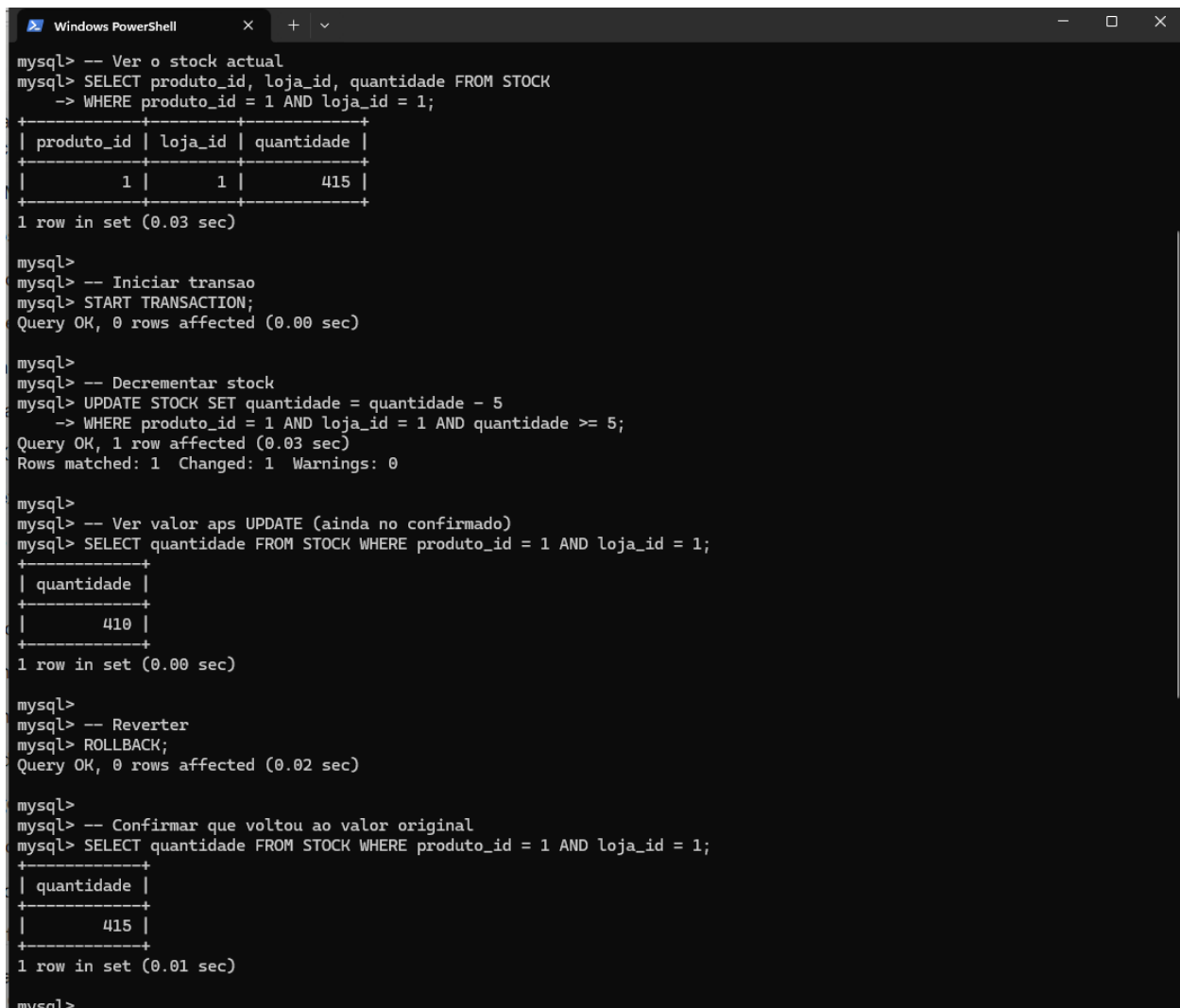
Se o Master cair permanentemente, os Slaves continuam a responder a leituras mas nenhuma escrita é possível no sistema. Para tornar o sistema mais resiliente, o grupo poderia adoptar uma estratégia de **failover**: promover manualmente o Slave mais actualizado (menor Seconds_Behind_Master) a novo Master, e reconfigurar os restantes Slaves para apontar para ele. Em ambientes de produção isso é automatizado com ferramentas como o **MySQL Router** ou **Orchestrator**, que detectam a falha e promovem o novo Master sem intervenção manual.

5. Fase 4 — Transações, ACID e Two-Phase Commit

5.1 Transação Local no MySQL

Comandos executados (nó Luanda):

```
START TRANSACTION;
UPDATE STOCK SET quantidade = quantidade - 5
  WHERE produto_id=1 AND loja_id=1 AND quantidade>=5;
SELECT quantidade FROM STOCK WHERE produto_id=1 AND loja_id=1;
ROLLBACK;
-- repetir com COMMIT
```



```
mysql> -- Ver o stock actual
mysql> SELECT produto_id, loja_id, quantidade FROM STOCK
  -> WHERE produto_id = 1 AND loja_id = 1;
+-----+-----+-----+
| produto_id | loja_id | quantidade |
+-----+-----+-----+
|          1 |          1 |          415 |
+-----+-----+-----+
1 row in set (0.03 sec)

mysql>
mysql> -- Iniciar transao
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Decrementar stock
mysql> UPDATE STOCK SET quantidade = quantidade - 5
  -> WHERE produto_id = 1 AND loja_id = 1 AND quantidade >= 5;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Ver valor aps UPDATE (ainda no confirmado)
mysql> SELECT quantidade FROM STOCK WHERE produto_id = 1 AND loja_id = 1;
+-----+
| quantidade |
+-----+
|          410 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Reverter
mysql> ROLLBACK;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Confirmar que voltou ao valor original
mysql> SELECT quantidade FROM STOCK WHERE produto_id = 1 AND loja_id = 1;
+-----+
| quantidade |
+-----+
|          415 |
+-----+
1 row in set (0.01 sec)

mysql>
```

Registo de Resultados — FASE-4A (Transação Local)

Item	Resultado
Quantidade ANTES do UPDATE	415

Quantidade APÓS UPDATE (antes do ROLLBACK)	410
Quantidade APÓS ROLLBACK (igual ao inicial?)	SIM - 415
Quantidade APÓS COMMIT	410

5.2 Transação Distribuída com Python (2PC Simplificado)

Foi adaptado e executado o script `scripts/transacao.py`, que implementa um Two-Phase Commit simplificado entre os nós Luanda e Benguela: decremento de stock em Benguela e registo da venda em Luanda, com COMMIT/ROLLBACK coordenados nos dois nós.

Comandos executados:

```
pip install mysql-connector-python
python scripts/transacao.py
```

```

Windows PowerShell
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> python -m pip --version
pip 26.1.2 from C:\Users\Nzola Kiampava\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\pip (python 3.14)
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> python -m pip install mysql-connector-python
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.7.0-cp314-cp314-win_amd64.whl.metadata (11 kB)
  Downloading mysql_connector_python-9.7.0-cp314-cp314-win_amd64.whl (18.2 MB)
    18.2/18.2 MB 550.9 kB/s 0:00:34
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.7.0
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> python scripts/transacao.py
Stock em Benguela ANTES: 207
Vendas em Luanda ANTES: 30000
[14:43:30] [Benguela ] UPDATE STOCK          -> OK
[14:43:31] [Luanda   ] INSERT VENDA           -> OK
[14:43:31] [Benguela ] COMMIT                 -> OK
[14:43:31] [Luanda   ] COMMIT                 -> OK
✓ Transação concluída com sucesso.
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza>

```

Registo de Resultados — FASE-4B (Transação Distribuída)

Item	Resultado
Stock em Benguela ANTES da transação	207
Stock em Benguela APÓS a transação (sucesso)	197
N.º de vendas em Luanda ANTES	30000
N.º de vendas em Luanda APÓS (sucesso)	30001
Resultado com erro forçado — ROLLBACK executado? (S/N)	S
Stock em Benguela após ROLLBACK (igual ao inicial?)	197

Evidências

```

Windows PowerShell
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadokwanza -e
"SELECT COUNT(*) FROM VENDA;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| COUNT(*) |
+-----+
|      30001 |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-luanda
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> docker exec -it no-luanda mysql -uroot -pkwanza2024 mercadokwanza -e
"SELECT COUNT(*) FROM VENDA;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| COUNT(*) |
+-----+
|      30001 |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-luanda
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza> docker exec -it no-benguela mysql -uroot -pkwanza2024 mercadokwanza
-e "SELECT quantidade FROM STOCK WHERE produto_id=5 AND loja_id=6;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+-----+
| quantidade |
+-----+
|          197 |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug no-benguela
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Nzola Kiampava\Documents\mercadokwanza>

```

Registo de Uso de IA — Fase 4 — Apoio na implementação/adaptação do script transacao.py

Ferramenta usada:

CLAUDE SONNET 4.6

Prompt submetido (resumo):

Criação do script `transacao.py` para implementar um Two-Phase Commit simplificado entre os nós Luanda (porta 3310) e Benguela (porta 3307), com decremento de stock em Benguela e registo de venda em Luanda, incluindo tratamento de erros e ROLLBACK coordenado.

O que a IA respondeu (resumo):

Script Python completo com função de log com timestamp, ligação aos dois nós com `autocommit=False`, verificação de stock antes da operação, UPDATE em Benguela com FOR UPDATE, INSERT de VENDA e ITEM_VENDA em Luanda, COMMIT nos dois nós em caso de sucesso e ROLLBACK nos dois nós em caso de erro.

O que o grupo corrigiu, melhorou ou rejeitou:

Foi necessário ajustar a porta do nó Luanda de 3306 para 3310 (devido ao conflito com o MySQL local). O teste de erro simulado foi adicionado manualmente inserindo um `raise Exception` antes do INSERT em Luanda para verificar o comportamento do ROLLBACK.

Reflexão Escrita — Fase 4

Pergunta 1: O protocolo Two-Phase Commit que implementaram é simplificado — não trata todos os casos de falha. O que aconteceria se o nó Benguela fizesse COMMIT mas a ligação falhasse antes de o nó Luanda fazer o seu COMMIT? Este estado chama-se *'in-doubt transaction'*. Como poderia ser resolvido?

Se o nó Benguela fizesse COMMIT mas a ligação falhasse antes do COMMIT em Luanda, o sistema ficaria num estado inconsistente — o stock em Benguela já foi decrementado definitivamente, mas a venda em Luanda nunca foi registada. Este estado chama-se *in-doubt transaction*. No protocolo 2PC simplificado que

implementámos não existe mecanismo para resolver isto automaticamente. Numa implementação real, a solução seria usar um **log de transações persistente**: cada nó regista a sua intenção de COMMIT antes de o executar, permitindo que um coordenador externo consulte esse log após a recuperação e force o COMMIT ou ROLLBACK nos nós que ficaram em estado indeterminado.

Pergunta 2: Qual a diferença entre o nível de isolamento *READ COMMITTED* e *SERIALIZABLE*? Para um sistema de stock em tempo real, qual escolheriam e porquê?

Com **READ COMMITTED**, uma transação só vê dados que já foram confirmados por outras transações — evita leituras sujas, mas permite que a mesma query executada duas vezes dentro da mesma transação retorne valores diferentes (leitura não repetível). Com **SERIALIZABLE**, as transações são executadas como se fossem em série — nenhuma interferência é possível, mas o custo em bloqueios e desempenho é muito maior. Para um sistema de stock em tempo real como o MercadoKwanza, escolheríamos **READ COMMITTED** — é o equilíbrio adequado entre consistência e desempenho, evitando leituras sujas sem bloquear excessivamente o sistema em momentos de alto volume de vendas.

Pergunta 3: O uso de 'FOR UPDATE' no SELECT em Benguela bloqueou o registo. Que problema resolve este bloqueio? Que problema pode criar se muitas transações tentarem aceder ao mesmo produto em simultâneo?

O FOR UPDATE no SELECT bloqueia o registo de stock em Benguela durante toda a transação, impedindo que outra transação leia e altere a mesma linha ao mesmo tempo. Resolve o problema de **vendas a descoberto** — sem este bloqueio, duas transações poderiam ler o mesmo stock disponível (ex: 10 unidades) e ambas prosseguir, resultando em stock negativo. O problema que pode criar é **contenção**: se muitas transações tentarem aceder ao mesmo produto simultaneamente, ficam todas em fila à espera que o bloqueio seja libertado, aumentando a latência e podendo causar timeouts em períodos de alto volume.

6. Registo Consolidado de Uso de IA

A tabela seguinte resume todas as utilizações de IA ao longo da Parte 1, com a ferramenta usada, o prompt submetido (resumo), a resposta obtida (resumo) e as correcções efectuadas pelo grupo.

Fase	Ferramenta	Prompt (resumo)	Correcções feitas pelo grupo
Fase 0	Claude sonnet 4.6	Geração do script mercadokwanza.sql completo para MySQL 8.0 com 7 tabelas, stored procedures e dados angolanos realistas (3 províncias, 15 lojas, 200 produtos, 5000 clientes, 30000 vendas)	O script inicial causou ERROR CODE 2013 (Lost Connection) por usar loops com SELECTs individuais. Foi reescrito substituindo os loops por INSERT...SELECT em lote com CROSS JOIN, reduzindo o tempo de execução para menos de 2 minutos
Fase 1	Claude sonnet 4.6	Criação do docker-compose.yml com 4 contentores (3 MySQL + 1 MongoDB), estrutura de pastas do projecto e comandos para levantar o cluster	Foi necessário alterar a porta do nó Luanda de 3306 para 3310 devido a conflito com o MySQL instalado localmente no Windows. O comando de substituição PowerShell falhou no CMD e o ficheiro foi corrigido manualmente via download
Fase 3		Configuração da replicação MySQL Master-Slave com CHANGE MASTER TO e resolução do erro de autenticação caching_sha2_password	Foi criado um utilizador dedicado replicador com mysql_native_password para contornar o erro de SSL. Foi também necessário fazer dump e reimportação dos dados nos Slaves pois a replicação não copia dados históricos
Fase 4	Claude sonnet 4.6	Criação do script transacao.py para Two-Phase Commit simplificado entre Luanda (porta 3310) e Benguela (porta 3307)	Foi ajustada a porta do nó Luanda de 3306 para 3310. O teste de erro simulado foi adicionado manualmente com raise Exception antes do INSERT

		com log de operações e tratamento de erros	em Luanda para verificar o ROLLBACK
--	--	--	-------------------------------------

7. Dificuldades Encontradas

1. ERROR CODE 2013 — Lost Connection (Fase 0)

Ao executar o script `mercadokwanza.sql`, a stored procedure de inserção das vendas causou perda de ligação por excesso de tempo. O loop de 30 000 iterações executava um SELECT individual em cada ciclo, tornando o processo demasiado lento. Foi resolvido reescrevendo o script com INSERT...SELECT em lote usando CROSS JOIN sobre tabelas temporárias de sequência.

2. Conflito de porta 3306 (Fase 1)

Ao levantar o cluster com `docker compose up -d`, o contentor no-luanda falhou porque a porta 3306 já estava ocupada pelo MySQL instalado localmente no Windows. Foi resolvido alterando o mapeamento para 3310:3306 no ficheiro `docker-compose.yml`.

3. Erro de autenticação na replicação — caching_sha2_password (Fase 3)

Ao configurar a replicação com CHANGE MASTER TO usando o utilizador root, o Slave Benguela apresentou o erro Authentication requires secure connection. O MySQL 8.0 usa por defeito o plugin caching_sha2_password que exige SSL. Foi resolvido criando um utilizador dedicado replicador com `mysql_native_password`.

4. Slaves sem dados históricos (Fase 3)

Após configurar a replicação, os nós Benguela e Huambo não tinham as tabelas nem os dados porque a replicação só copia operações futuras a partir do momento da configuração. Foi necessário exportar os dados do Master com `mysqldump` e importar manualmente nos dois Slaves.

5. Comando PowerShell vs CMD (Fases 1 e 3)

Alguns comandos fornecidos para CMD não funcionaram no PowerShell (como `2>nul` e `-replace`). Foi necessário adaptar os comandos para o ambiente PowerShell ou editar os ficheiros manualmente.